



Trailhead



[Apex & .NET Basics \(/content/learn/modules/apex_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/apex_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)



Use Asynchronous Apex ▾

Use Asynchronous Apex

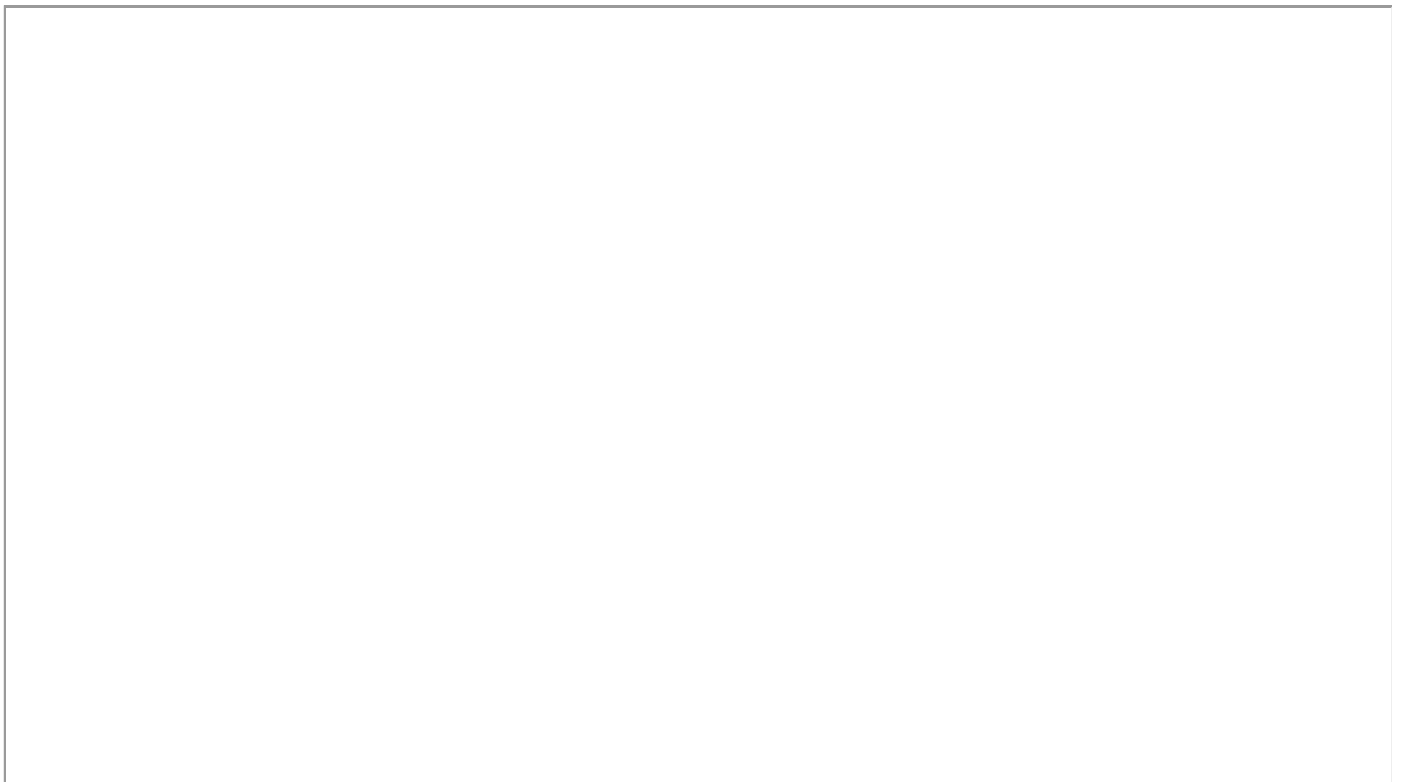
Learning Objectives

After completing this unit, you'll be able to:

- Know when to use Asynchronous Apex
- Use future methods to handle a web callout
- Work with the batchable interface to process a large number of records
- Understand the advantages of using the queueable interface when you need to meet in the middle

Follow Along with Trail Together

Want to follow along with an expert as you work through this step? Take a look at this video, part of the Trail Together series.



(This clip starts at the 32:07 minute mark, in case you want to rewind and watch the beginning of the step again.)

When to Go Asynchronous

As a .NET developer, you probably have already been exposed to the world of asynchronous programming. We'll assume that you know what it is and understand the benefits to using this type of programming model. What you might not know is when to use asynchronous programming on the Lightning Platform.

The following three reasons are usually behind choosing asynchronous programming.





triggers can't make callouts directly.

- **Creating a better and faster user experience** by offloading some processing to asynchronous calls. Why do Apex & .NET Basics (/content/learn/modules/apex_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer) everything at once? If it can wait, let it.

Use Asynchronous Apex

Future Methods

In situations where you need to make a callout to a web service or want to offload simple processing to an asynchronous task, creating a future method could be the way to go.

Changing a method from synchronous to asynchronous processing is amazingly easy. Essentially, you just add the

`@future` annotation to your method. Other than that, just make sure that the method is static and returns only a void type. For example, to create a method for performing a web service callout, we could do something like this:

```
1 public class MyFutureClass {
2     // Include callout=true when making callouts
3     @future(callout=true)
4     static void myFutureMethod(Set<Id> ids) {
5         // Get the list of contacts in the future method since
6         // you cannot pass objects as arguments to future methods
7         List<Contact> contacts = [SELECT Id, LastName, FirstName, Email
8             FROM Contact WHERE Id IN :ids];
9         // Loop through the results and call a method
10        // which contains the code to do the actual callout
11        for(Contact con: contacts) {
12            String response = anotherClass.calloutMethod(con.Id,
13                con.FirstName,
14                con.LastName,
15                con.Email);
16            // May want to add some code here to log
17            // the response to a custom object
18        }
19    }
20 }
```



We could then call it just like we would any other static method. Easy peasy!

Future Limitations

Future methods offer a great option for Salesforce developers, however, they're not without drawbacks. Here are some limitations to consider before using a future method.

- You can't track execution because no Apex job ID is returned.
- Parameters must be primitive data types, or collections of primitive data types. Future methods can't take sObjects as arguments as they might change in the time before the `@future` method executes.
- You can't chain future methods and have one call another. Use Queueable apex if you need execution in a certain order.

Although asynchronous calls are sometimes done to avoid limits, you still need to consider limits. Check out the link concerning Execution Governors and Limits in Resources.



particular time.

To use it, your class implements the `Database.Batchable` interface. You also define `start()`, `execute()`, and `finish()` methods. You can then invoke a batch class using the `Database.executeBatch` method. For example, the following code creates a batchable class that processes all accounts in an org and then sends an email when it is done.

Use Asynchronous Apex

```
1 global class MyBatchableClass implements
2     Database.Batchable<sObject>,
3     Database.Stateful {
4     // Used to record the total number of Accounts processed
5     global Integer numOfRecs = 0;
6     // Used to gather the records that will be passed to the interface method
7     // This method will only be called once and will return either a
8     // Database.QueryLocator object or an Iterable that contains the records
9     // or objects passed to the job.
10    global Database.QueryLocator start(Database.BatchableContext bc) {
11        return Database.getQueryLocator('SELECT Id, Name FROM Account');
12    }
13    // This is where the actual processing occurs as data is chunked into
14    // batches and the default batch size is 200.
15    global void execute(Database.BatchableContext bc, List<Account> scope) {
16        for(Account acc : scope) {
17            // Do some processing here
18            // and then increment the counter variable
19            numOfRecs = numOfRecs + 1;
20        }
21    }
```



Show More

You could then invoke the batch class using anonymous code such as this:

```
1 MyBatchableClass myBatchObject = new MyBatchableClass();
2 Database.executeBatch(myBatchObject);
```



We didn't specifically cover Scheduled Apex in this unit, but it's similar to the batchable interface. It implements the schedulable interface, and you can use it to invoke Apex at specific times. Learn more about it by checking out the [Asynchronous Apex](https://trailhead.salesforce.com/content/learn/modules/asynchronous_apex) (https://trailhead.salesforce.com/content/learn/modules/asynchronous_apex) module.

Batchable Limitations

The batchable interface is great, but as with just about everything, consider its limitations.

- Troubleshooting can be troublesome.
- Jobs are queued and subject to server availability, which can sometimes take longer than anticipated.
- Have we talked about limits yet?



Salesforce responded with Queueable Apex. It represents the best of future methods and the batchable interface, all rolled up into one super-duper asynchronous tool. Developers forced to use the slower batchable interface to get around limitations of future methods could now return to a tool that made more sense. Queueable Apex provides the following benefits to future methods.

Use Asynchronous Apex

- **Non-primitive types** - Classes can accept parameter variables of non-primitive data types, such as sObjects or custom Apex types.
- **Monitoring** - When you submit your job, a jobId is returned that you can use to identify the job and monitor its progress.
- **Chaining jobs** - You can chain one job to another job by starting a second job from a running job. Chaining jobs is useful for sequential processing.

So how does it work? Glad you asked.

Because Queueable Apex includes the best of future methods, it's much easier to implement than Batch Apex. It just doesn't have those pesky limitations we talked about. To demonstrate how it works, let's take the sample code that uses a future method to do a web callout and implement it using Queueable Apex.

```

1 public class MyQueueableClass implements Queueable {
2     private List<Contact> contacts;
3     // Constructor for the class, where we pass
4     // in the list of contacts that we want to process
5     public MyQueueableClass(List<Contact> myContacts) {
6         contacts = myContacts;
7     }
8     public void execute(QueueableContext context) {
9         // Loop through the contacts passed in through
10        // the constructor and call a method
11        // which contains the code to do the actual callout
12        for(Contact con: contacts) {
13            String response = anotherClass.calloutMethod(con.Id,
14                con.FirstName,
15                con.LastName,
16                con.Email);
17            // May still want to add some code here to log
18            // the response to a custom object
19        }
20    }
21 }
```



To invoke Queueable Apex, you need something like the following:

```

1 List<Contact> contacts = [SELECT Id, LastName, FirstName, Email
2     FROM Contact WHERE Is_Active__c = true];
3 Id jobId = System.enqueueJob(new MyQueueableClass(contacts));
```



Tell Me More

Along with queueable interfaces, Salesforce also introduced the Apex Flex Queue, which eliminated the limitation of five concurrent batches. It also allows developers to monitor and manage the order of queued jobs. Check the links in Resources for more info.



Resources

- [Trailhead: Asynchronous Apex \(https://trailhead.salesforce.com/content/learn/modules/asynchronous_apex\)](https://trailhead.salesforce.com/content/learn/modules/asynchronous_apex)
- [Apex Developer Guide: Execution Governors and Limits \(https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_gov_limits.htm\)](https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_gov_limits.htm)

Use Asynchronous Apex

Quiz

+100 Points

To complete this unit, you need to answer all the quiz questions correctly.

1 You can choose to use asynchronous processing when:

- ☐ A You need to make callouts to external web services.
- ☐ B You want to create a better and faster user experience.
- ☐ C You are processing a large number of records.
- ☐ D All of the above

2 Which of the following is NOT a type of asynchronous processing?

- ☐ A Batch Apex
- ☐ B Queueable Apex
- ☐ C Future Methods
- ☐ D Back to the Future Apex

Second attempt earns 50 points. Three or more earns 25 points.

Check the Quiz to Earn 100 Points



Trailhead Academy

Salesforce Admins

Career Paths

Code of Conduct

[Apex & .NET Basics \(/content/learn/modules/apex_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/apex_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

[Report Illegal Content \(EU\)](#)

[Use Asynchronous Apex](#)

Download Trailhead GO



© 2026 Salesforce, Inc. All rights reserved.

[Privacy Statement](#)

[Terms of Use](#)

[Use of Cookies](#)

[Trust](#)

[Accessibility](#)

[Cookie Preferences](#)

[Your Privacy Choices](#)

English

